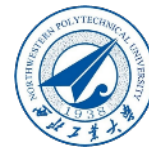


第4章 序列结构



- ◆ 掌握列表创建，以及列表元素操作
- ◆ 掌握元组的创建与删除，并区分元组与列表
- ◆ 掌握字典的创建，以及字典元素操作
- ◆ 掌握集合的创建，以及集合运算
- ◆ 了解序列运算，包括序列解包、序列元素的访问等
- ◆ 了解其它序列类型

- 在Python中，最基本的数据结构是序列。序列中的成员有序排列，都可以通过下标偏移量访问到它的一个或几个成员。除了前面已经介绍过的字符串，最常见的序列是列表和元组。
- 在本章中，除了列表和元组外，还要介绍另外两个序列：字典和集合。

- **序列**（sequence）是一种用来**存放多个值的**数据类型，**序列中对象类型可以相同也可以不同**。
- **序列中的每个元素**可以通过索引来进行访问（集合类型除外）。
- 序列按照其中的**元素是否可变**分为**可变序列**和**不可变序列**，按照**元素是否有序**分为**有序序列**和**无序序列**。

- 列表是Python内置的**有序、可变**序列，列表的所有元素放在一对中括号“**[]**”中，并使用逗号分隔开。
- 当**列表元素增加或删除**时，**列表对象自动进行扩展或收缩内存**，保证元素之间没有缝隙。
- 在Python中，一个**列表中的数据类型可以各不相同**，可以同时**为整数、实数、字符串**等基本类型，甚至是**列表、元组、字典、集合**以及其他**自定义类型的对象**。

下面几个都是合法的列表对象：

```
['Google', 'Baidu', 1997, 2008]
```

```
[1, 2, 3, 4, 5]
```

```
["a", "b", "c", "d"]
```

```
[123, ["das", "aaa"], 234]
```

- 列表是Python中最具灵活性的有序集合对象类型。和字符串不同的是，列表具有可变长度、异构以及任意嵌套列表的特点。
- 列表是可变对象，支持在原处修改。
- Python中列表存放的元素是值的引用，并不直接存储值，**列表类似于其它语言的数组。**
- **列表中的自动化的内存管理**可能会涉及列表中大量的元素移动，效率较低，因此应**尽量从列表尾部**进行元素的**添加与删除操作**，提高列表的工作效率。

1 . 列表的创建（赋值）

- 将列表元素放置在一对方括号[]内，以逗号分隔，将这个列表赋值给变量，即可创建列表。

```
In [10]: ▶ lst1 = ["cat", "dog", "tiger", "1024"]  
          lst2 = [31, 32, 101.45, "door"]  
          lst3 = [1, 2, ['hello', 5]]  
          print(lst1, lst2)  
          print(lst3)  
  
          ['cat', 'dog', 'tiger', '1024'] [31, 32, 101.45, 'door']  
          [1, 2, ['hello', 5]]
```

2 . 列表的创建 (list()函数)

- 也可以使用list()函数将元组、range对象、字符串或其它类型的可迭代对象类型的数据转换为列表。

```
In [11]: ▶ list1= list("abc")  
list2=list(range(1,10,2))  
print(list1)  
print(list2)
```

```
['a', 'b', 'c']  
[1, 3, 5, 7, 9]
```

3. 列表元素的访问与计数

- 列表中的元素可以使用下标操作符 `list[index]` 访问列表中下标为 `index` 的元素，`len(list)` 获取列表 `list` 的长度。
- `list[index]` 可以像变量一样使用，被称为下标变量。
- 当列表下标 `index` 为正整数时，表示从列表头开始计数， $0 \leq \text{index} \leq \text{len}(\text{list}) - 1$ 。当 `index` 为负整数时，表示从列表末尾开始计数， $-\text{len}(\text{list}) \leq \text{index} \leq -1$ ，将列表长度和负数下标相加就可以得到实际的位置。
- 当列表元素是序列时，可以采用二级下标方式 `list[index1][index2]`。

3. 列表元素的访问与计数示例：

```
In [22]: ▶ a = [1, 2, 3, 4, 5, 6, "abcd"]  
          print(len(a))  
          print(a[0] )  
          print(a[6][0])  
  
          7  
          1  
          a
```

3. 列表元素的访问与计数

- 使用列表对象的`index()`方法获取指定元素首次出现的下标，若列表对象中不存在指定元素，则抛出异常。

```
In [20]: ► aList=[3, 4, 5, 5.5, 7, 9, 11, 13, 15, 17]
          print(aList.index(7))
          print(aList.index(1))
```

4

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-20-5e5d901dcb30> in <module>
      1 aList=[3, 4, 5, 5.5, 7, 9, 11, 13, 15, 17]
      2 print(aList.index(7))
----> 3 print(aList.index(1))

ValueError: 1 is not in list
```

4. 改变列表元素值

➤ 可以通过列表的下标来修改列表里面的值，格式为：

变量名[下标] = 新的值

```
In [15]: ► x = [1, 1, 3, 4]
          x[1] = 2    #将列表中第二个1改为2
          print(x)

          [1, 2, 3, 4]
```

4. 改变列表元素值

➤ 列表元素分段改变:

```
In [16]: ▶ name = list('Perl')  
          name[1:] = list('ython')  
          print(name)
```

```
['P', 'y', 't', 'h', 'o', 'n']
```

5. 列表元素的增加

➤ 列表元素的添加可以使用 `append()`、`extend()` 和 `insert()`，或者使用 “+” 和 “*” 运算符。除了 “+” 运算符外，其它几个方法都属于原地操作。

➤ `append()` 方法

通过 `append()` 方法在当前列表尾部追加元素，是原地修改列表，速度较快。

➤ `extend()` 方法

使用列表对象的 `extend()` 方法可以将另一个迭代对象的所有元素添加至该列表对象尾部。通过 `extend()` 方法来增加列表元素也不改变其内存首地址，属于原地操作。

5. 列表元素的增加

➤ insert() 方法

使用列表对象的insert(index, x)方法可以将元素添加至列表的指定位置。

➤ 列表的insert()方法可以在列表的任意位置插入元素，但由于列表的自动内存管理功能，insert()方法会引起插入位置之后所有元素的移动，这会影响处理速度。

5. 列表元素的增加

① 采用+和*的方式

- 采用+和*的方式在形式上简单容易理解，但严格意义上并不是真的为列表添加元素，而是有可能会被处理为**创建一个新的列表**，并将原列表中的元素和新元素依次复制到新列表的内存空间。
- 由于涉及大量元素的复制，该操作**速度较慢**，对于列表元素较多和进行大量数据操作过程中从性能的考虑不建议使用该方法。

5. 列表元素的增加

① 采用+和*的方式:

```
s[下标] = d    # 设置元素，d为任意值。  
s += [d]       # 更新列表s，将列表d元素增加到列表s中。  
s *= n         # 更新列表s，其元素重复n次。
```

5. 列表元素的增加

① 采用+运算符:

- 如果是通过“+”运算符来增加列表元素，实际上是创建了一个新列表，并将原列表中的元素和新元素依次复制到新列表的内存空间。由于涉及大量元素的复制，该操作速度较慢，在涉及大量元素添加时不建议使用该方法。

```
In [25]: ► aList = [1, 2, 3]
          aList += [4, 5]
          print(aList)

          [1, 2, 3, 4, 5]
```

5. 列表元素的增加

② 采用*运算符:

- 使用乘法*来扩展列表对象，将列表与整数相乘，生成一个**新列表**，新列表是原列表中元素的重复。
- 当使用*运算符将包含列表的列表重复并创建新列表时，当修改其中一个值时，相应的引用也会被修改。

```
In [26]: ▶ bList = [[1, 2, 3]] * 3  
          bList[0][0] = 4  
          print(bList)
```

```
[[4, 2, 3], [4, 2, 3], [4, 2, 3]]
```

5. 列表元素的增加

② 采用列表对象的**append()**方法:

- **append()**方法是将所**附加的内容**作为一个**元素**来处理，附加的内容是一个含有多个元素的列表时，相当于将该列表作为一个**元素**附加到原有列表的尾部。
- 通过**append()**方法在当前列表尾部追加元素，是**原地修改**列表，**速度**较快。

```
In [34]: ► aList = [3, 4, 5, 6]
          aList.append(7)
          print(aList)
          aList.append([3, 2])
          print(aList)
```

```
[3, 4, 5, 6, 7]
[3, 4, 5, 6, 7, [3, 2]]
```

5. 列表元素的增加

② 采用列表对象的**extend()**方法:

- 使用列表对象的**extend()**方法可以将另一个迭代对象的所有元素添加至该列表对象尾部。通过**extend()**方法来增加列表元素也不改变其内存首地址, 属于**原地操作**。

```
In [30]: ► aList = [1, 2, 3, 4]
          aList.extend([5, 6])
          print(aList)

          [1, 2, 3, 4, 5, 6]
```

5. 列表元素的增加

② 采用列表对象的insert()方法:

- 使用列表对象的insert(index, object)方法将元素添加到指定位置。第一个参数index表示插入的位置，第二个参数表示要插入的对象
- 列表的insert()方法可以在列表的任意位置插入元素，但由于列表的自动内存管理功能，insert()方法会引起插入位置之后所有元素的移动，这会严重影响处理速度。

```
In [35]: ▶ a=[1, 2, 3, 5]
          a.insert(3, 6)      #在下标为3的位置插入元素6
          print(a)

          [1, 2, 3, 6, 5]
```

6. 列表元素的删除

可以采用以下三种形式。

- 使用**del**命令删除列表中指定位置的元素。如同一般的变量，**del**命令也可以删除整个列表变量。
- 使用列表的**pop**方法删除并返回指定位置（默认为最后一个）上的元素。若指定的索引超出列表的范围，会抛出异常。
- 使用列表的**remove()**方法删除首次出现的元素。若没有此元素，会抛出异常。

6. 列表元素的删除

```
In [43]: ▶ names = ['one', 'two', 'three', 'four', 'five', 'six']  
del names[1]      #删除names的第二个元素  
names[1:4]=[]     #删除names的第二至第四个元素  
print(names)  
  
['one', 'six']
```

6. 列表元素的删除

```
In [2]: ▶ list3= [1,2,3]
list3.pop(1)
list3.pop()
print(list3)
del list3
print(list3)
```

```
[1]
```

```
NameError                                Traceback (most recent call last)
<ipython-input-2-11b44e9238c1> in <module>
      4 print(list3)
      5 del list3
----> 6 print(list3)

NameError: name 'list3' is not defined
```

6. 列表元素的排序和反转

- 列表结构具有对象自身提供的**sort()**方法进行原地排序，即这种排序是**永久性排序**，将会按照顺序改变列表中元素的次序。
- 若要按与字母顺序相反的顺序排列列表元素，只需向**sort()**方法传递参数 **reverse=True**。

```
In [58]: ▶ cars = [ 'bmw', 'audi', 'toyota', 'subaru' ]
cars.sort()
print (cars)                                # 改变了列表中元素的顺序
cars.sort (reverse = True)
print (cars)                                # 改变了列表中元素的顺序

['audi', 'bmw', 'subaru', 'toyota']
['toyota', 'subaru', 'bmw', 'audi']
```

6. 列表元素的排序和反转

- 要**保留**列表元素**原来**的**排列顺序**，同时以特定的顺序呈现它，可使用内置函数**sorted()**。

sorted(iterable, key = None, reverse = False)

返回系列的排序列表。

- 函数**sorted()**让你能够按特定顺序显示列表元素，同时不影响它在列表中的原始排序。

```
In [62]:  cars = [ 'bmw', 'audi', 'toyota', 'subaru' ]  
          print(sorted(cars))  
          print (cars)                                # 并没有实际改变列表中元素的顺序  
          s3 = ' abAC '  
          sorted(s3, key = str.lower)
```

```
['audi', 'bmw', 'subaru', 'toyota']  
['bmw', 'audi', 'toyota', 'subaru']
```

```
Out[62]: [' ', ' ', 'a', 'A', 'b', 'C']
```

6. 列表元素的排序和反转

- **sorted()**排序后会生成**新列表**，**原列表不会改变**。
- **reverse()**方法用于将列表所有元素逆序或翻转。

In [6]: ▶ a=[15, 8, 14, 16, 10, 19, 13, 18, 11, 17, 12, 9]

```
print(id(a))  
b = sorted(a)  
print(b)  
print(id(b))  
a.sort(key=str)           #原地排序，指定排序依据根据字符串大小  
print(a)  
print(id(a))  
a.reverse()               #反转列表，原地操作  
print(a)  
print(id(a))
```

```
1806003938112  
[8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]  
1806003335232  
[10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 8, 9]  
1806003938112  
[9, 8, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10]  
1806003938112
```

7. 列表切片操作

- 使用语法 “**list_name[start:end:step]**”来返回列表list_name的一个片段。**start**表示切片开始位置，默认为0；**end**表示切片截止（但不包含）位置，默认为列表长度；**step**表示切片的步长，默认为1。
- 当**start**为0时可以省略，当**end**为列表长度时可以省略，当**step**为1时可以省略，省略步长时还可以同时省略最后一个冒号。
- 当**step**为负整数时，表示反向切片，这时**start**位置应该在**end**位置的右侧，否则会返回空列表，**start**默认为-1，**end**默认为列表第1个元素前面的位置。-1表示列表最后一个元素的位置，其它以此类推。

7. 列表切片操作示例:

```
In [9]: ► numbers = [1, 2, 3, 4, 5, 6, 7, 8]
print(numbers[:])           #返回包含原列表中所有元素的新列表
print(numbers[:2])          #从第1个元素开始, 隔1个取元素
print(numbers[::-1])        #反向切片
print(numbers[1:3])          #指定切片开始位置和结束位置, 步长默认为1
print(numbers[1::2])         #指定切片开始位置和步长, 省略结束位置

[1, 2, 3, 4, 5, 6, 7, 8]
[1, 3, 5, 7]
[8, 7, 6, 5, 4, 3, 2, 1]
[2, 3]
[2, 4, 6, 8]
```

7. 列表切片操作

使用切片获取列表的部分元素

- 使用切片可以返回列表中部分元素组成的新列表。切片操作不会因为下标越界而抛出异常，而是简单地在列表尾部截断或者返回一个空列表，代码具有更强的健壮性。

使用切片为列表增加/删除元素

- 可以使用切片操作在列表任意位置插入新元素，但是并不影响列表对象的内存地址，属于原地操作。

7. 列表切片操作示例：

```
In [10]: ▶ num = [1, 2, 3, 4, 5]
num[len(num):] = [6, 7, 8]
print(num)
num[:0] = [-2, -1]
print(num)
aList = [3, 5, 7, 9]
aList[:3] = []                                     #删除列表中前3个元素
print(aList)
```

[1, 2, 3, 4, 5, 6, 7, 8]
[-2, -1, 1, 2, 3, 4, 5, 6, 7, 8]
[9]

8. 列表成员测试

- 使用`in/not in`运算符可以判断一个元素是否在列表中。

```
In [11]: ▶ aList = [1, 2, 3]
          a = 2
          print(a in aList)
          print(4 not in aList)

True
True
```

9. 列表的比较

- 关系运算符（<、>、==、!=、<=、>=）也可以用来对列表进行比较。

```
In [12]: ► list1 = [1, 2, 3]
          list2 = [2, 5, 6]
          print(list1 > list2)
          print(list1 < list2)
```

```
False
True
```

9. 列表中的元素随机排列shuffle() 函数

- **shuffle()函数：** random模块中的**shuffle()**函数可实现随机排列列表中的元素。

```
In [13]: ▶ import random                                #导入模块  
list1=[2,3,7,1,6,12]  
random.shuffle(list1)  
list1
```

```
Out[13]: [6, 12, 2, 7, 3, 1]
```

在**Python**中字符串、列表和以及元组都是序列类型。

- 所谓序列，即成员有序排列，并且可以通过偏移量访问到它的一个或者几个成员。
- 序列中的每个元素都被分配一个数字——它的位置，也称为索引，第一个索引是**0**，第二个索引是**1**，依此类推。
- 序列都可以进行的操作包括索引、切片、加、乘、检查成员。
- **Python**已经内置确定序列的长度以及确定最大和最小的元素的方法。

列表是一种序列类型，序列的常用操作见表

操作	描述
x in s	如果元素x在序列s中则返回True
x not in s	如果元素x不在序列s中则返回True
s1+s2	连接两个序列s1和s2，得到一个新序列
s*n, n*s	序列s复制n次得到一个新序列
s[i]	得到序列s的第i个元素
s[i:j]	得到序列s从下标i到j-1的片段
len(s)	返回序列s包含的元素个数
max(s)	返回序列s的最大元素
min(s)	返回序列s的最小元素
sum(x)	返回序列s中所有元素之和
<, <=, >, >=, ==, !=	比较两个序列

列表对象常用的方法见表

方 法	描述
list.append(x)	在列表list末尾添加新的对象x
list.count(x)	返回x在列表list中出现的次数
list.extend(seq)	在列表list末尾一次性追加seq序列中的所有元素
list.index(x)	返回列表list中第一个值为x的元素的下标，若不存在抛出异常
list.insert(index, x)	在列表list中index位置处添加元素x
list.pop([index])	删除并返回列表指定位置的元素，默认为最后一个元素
list.remove(x)	移除列表list中x的第一个匹配项
list.reverse()	反向列表list中的元素
list.sort(key=None, reverse=None)	对列表list进行排序，key参数的值为一个函数，此函数只有一个参数且返回一个值，此函数将在每个元素比较前被调用，reverse表示是否逆序
list.clear()	删除列表list中的所有元素，但保留列表对象
list.copy()	用于复制列表，返回复制后的新列表

可以利用 `dir()` 来查看列表的方法:

```
Python 3.7.6 Shell
File Edit Shell Debug Options Window Help
Python 3.7.6 (tags/v3.7.6:43364a7ae0, Dec 19 2019, 00:42:30) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> dir(list)
['_add_', '__class__', '__contains__', '__delattr__', '__delitem__', '__dir__',
, '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__getitem__',
, '__gt__', '__hash__', '__iadd__', '__imul__', '__init__', '__init_subclass__',
, '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__r
educe__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__',
, '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear',
, 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort
,']
>>> |
```

- 列表的功能虽然很强大，但负担也很重，在很大程度上影响了运行效率。有时候我们并不需要那么多功能，很希望能有个轻量级的列表，元组（tuple）正是这样一种类型。
- 列表和元组都属于有序序列，都支持使用双向索引访问其中的元素。
- 元组属于**不可变**（immutable）序列，一旦被创建，元组内的元素**不可以修改**，也**无法添加或删除**元素，若要调整，只能重新建立元组。
- 元组没有提供append()、extend()和insert()等方法，无法向元组中添加元素；同样，元组也没有remove()和pop()方法，也不支持对元组元素进行del操作，不能从元组中删除元素，而只能使用del命令删除整个元组。

- 虽然tuple的元素不可改变，但它可以包含可变的对象，比如list列表，可改变元组中可变对象的值。

```
In [12]: ► tuple4 = ('a', 'b', ['A', 'B'])  
          tuple4[2][0] = 'X'  
          tuple4[2][1] = 'Y'  
          tuple4[2][2:] = 'Z'  
          print(tuple4)
```

```
('a', 'b', ['X', 'Y', 'Z'])
```

- 元组也支持切片操作，但是**只能通过切片来访问**元组中的元素，而**不允许使用切片来修改**元组中元素的值，也不支持使用切片操作来为元组增加或删除元素。
- Python内部实现对**元组**做了大量优化，**访问速度比列表更快**。如果定义了一系列常量值，主要用途仅是对它们进行遍历或其他类似用途，而不需要对其元素进行任何修改，那么一般建议使用元组而不用列表。

- 元组属于不可变序列。采用圆括号中用逗号分隔的方式进行定义，圆括号可以省略。
- 也可以使用`tuple()`函数将元组、`range`对象、字符串或其他类型的可迭代对象类型的数据转换为元组。
- 可用索引访问元组元素。
- 与列表相同，元组中的元素也可以为基本数据类型或者列表、元组等序列类型。
- 若要创建只包含一个元素的元组，要在元素后加一个逗号。
- 当不再使用时，使用`del`命令删除整个元组，但不能删除元组的元素。

```
In [6]: ► a = (1, 'a', [2, 3])
        b = tuple('abc')
        color_tuple = 'Red', 'Green', 'Blue'    #元组中只有一个元素，必须在后面多写一个逗号
        t2 = (4)                                #和t2 = 4是一样的
        t3 = (4,)                              #元组中只有一个元素，必须在后面多写一个逗号
        t4 = tuple(range(5))                   #将其他迭代对象转换为元组
        print(a)
        print(b)
        print(color_tuple)
        print(t2)
        print(t3)
        print(t4)

(1, 'a', [2, 3])
('a', 'b', 'c')
('Red', 'Green', 'Blue')
4
(4,)
(0, 1, 2, 3, 4)
```

元组的基本操作：

- 元组也是序列，因此一些用于列表的基本操作也可以用在元组上。
- 可以使用下标访问元组的元素，支持两个方法 `count()` 和 `index()`，提供 `in` 和 `not in` 运算符来判断元素是否在元组中，提供对元组进行切片等等。

元组的基本操作:

```
In [10]: ▶ t = (1, 2, 3)
          tt = tuple(range(4, 7))
          print("The second element in t is %d"%t[1])
          ttt = t + tt           #元组连接
          print(ttt)
          tttt = ttt[2:7]        #元组切片
          print(tttt)
          print(tttt.count(2))   #计算指定元素出现的次数
          print(tttt.index(5))   #计算指定元素第1次出现的下标
          t2=(1, 2, 3) * 3       #元组可以数乘
          print(t2)
```

```
The second element in t is 2
(1, 2, 3, 4, 5, 6)
(3, 4, 5, 6)
0
2
(1, 2, 3, 1, 2, 3, 1, 2, 3)
```

- 采用 `list()` 函数可以将元组转换为列表。
- 采用 `tuple()` 函数可以将列表转换为元组。

```
In [13]: ► a = tuple([1, 3, 5])  
          b = list((1, 3, 5))  
          print(a)  
          print(b)
```

```
(1, 3, 5)  
[1, 3, 5]
```

- 元组有很多用途，例如：坐标(x, y)，数据库中的员工记录等等。
- 元组和字符串一样，不可改变，即不能给元组的一个独立的元素赋值。
- 元组和列表看起来不同的一点是：元组用的是圆括号（），而列表用的是方括号[]。

```
In [14]: ▶ tup = tuple('bar') #创建元组
print('输出元组tup: ', tup) #输出元组
nested_tup = (4, 5, 6), (7, 8)
print('输出元组tup: ', nested_tup) #输出元素是元组的元组
print('元组的连接', tup+tuple('wwy'))
a, b, c = tup #元组的拆分
print(a, b, c)
print(tup.count(a)) # 统计某个数值在元组中出现的次数
```

```
输出元组tup: ('b', 'a', 'r')
输出元组tup: ((4, 5, 6), (7, 8))
元组的连接 ('b', 'a', 'r', 'w', 'w', 'y')
b a r
1
```

元组与列表的区别



- 元组与列表都是序列类型的一种扩展。元组 (tuple) 是一组有序系列，包含0个或多个对象引用。
- 元组是不可变的对象，即不能修改、添加或删除元组中的元素，但可以通过索引访问元组中的元素。
- 像列表一样，也可以用for 循环来遍历元组中的所有值。
 - 元组的访问和处理速度比列表更快，如果主要用途是对元素遍历或查找，而不需要对元素进行修改，一般建议使用元组而不是列表。
 - 另一方面，由于元组的内在实现不允许修改其元素值，从而适用于在一些场合中进行数据的保护。

➤字典是一种通过名字或者关键字引用的数据结构，其键可以是数字、字符串、元组，这种结构类型也称之为映射。字典类型是Python中唯一内建的映射类型。

➤字典，也称映射，是一个由**键/值对**组成的**非排序可变集合体**。键值对在字典中以下面的方式标记：

```
dict = {key1 : value1, key2 : value2 }
```

➤**键/值对**用**冒号**分割，而各个**元素之间**用**逗号**分割，所有**元素都****包括在花括号**中。

- 字典中元素的“键”可以是Python中任意不可变数据，例如整数、实数、复数、字符串、元组等可哈希数据，但不能使用列表、集合、字典或其他可变类型作为字典的“键”。
- 在同一个字典中，“键(key)”必须是唯一的，但“值(value)”是可以重复的。
- 列表是有序的对象集合，字典是无序的对象集合。两者之间的区别在于：字典当中的元素是通过键来存取的，而不是通过偏移存取。

1. 字典的创建:

① 直接利用大括号 {} 内添加键值对的方式进行内容的赋值。

```
In [15]: ► d={'one':1,'two':2,'three':3}
          print(d)
          dict1 = {'Alice': '2341', 'Beth': '9102', 'Cecil': '3258'}
          dict1['Jack'] = '1234'                                #为字典添加元素
          print(dict1)                                         # 输出完整的字典
          type(dict1)

{'one': 1, 'two': 2, 'three': 3}
{'Alice': '2341', 'Beth': '9102', 'Cecil': '3258', 'Jack': '1234'}
```

Out[15]: dict

1. 字典的创建:

② 通过内置的dict()函数来创建。

```
In [17]: ► lot = [(1, 'a'), (2, 'b'), (3, 'c')]          #定义一个列表
          dict1= dict(lot)                               #使用dict类转换列表到字典
          print(dict1)

          {1: 'a', 2: 'b', 3: 'c'}
```

1. 字典的创建:

③ 通过关键字创建字典。

```
In [20]: ▶ d=dict(one=1, two=2, three=3)
          print(d)
          print(d['one'])
          print(d['three'])
```

```
{'one': 1, 'two': 2, 'three': 3}
1
3
```

2. 字典的基本操作:

① 字典元素的访问:

- 使用“**键**”作为下标就可以访问对应的“**值**”，如果字典中**不存在**这个“**键**”会**抛出异常**。
- 还可以通过`get()`方法返回**指定“键”对应的“值”**，并且允许指定该键不存在时返回特定的“值”。
- 使用`keys()`方法得到**所有的“键”**，使用`values()`方法得到**所有的“值”**，使用`item()`方法得到**所有的“键”和“值”**。

2. 字典的基本操作:

① 字典元素的访问

```
In [22]: ▶ aDict = {'年龄': 19, '成绩': [85, 90, 68, 72], '姓名': '王宁', 'sex': '男'}
print(aDict["年龄"])      #访问“键”对应的“值”
print(aDict.get('name', 'Not Exists.'))
print(aDict.keys())       #访问所有“键”
print(aDict.values())     #访问所有“值”
print(aDict.items())      #返回整个字典内容

19
Not Exists.
dict_keys(['年龄', '成绩', '姓名', 'sex'])
dict_values([19, [85, 90, 68, 72], '王宁', '男'])
dict_items([('年龄', 19), ('成绩', [85, 90, 68, 72]), ('姓名', '王宁'), ('sex', '男')])
```

2. 字典的基本操作:

② 字典元素的添加

➤ 使用字典的`update()`方法来添加元素，还可以使用“键”的方式来增加元素。

```
In [23]: aDict = {'年龄': 19, '成绩': [85, 90, 68, 72], '姓名': '王宁', 'sex': '男'}  
         aDict['年龄'] = 20  
         aDict['地址'] = "西安市长安区西京路1号"    #使用键的方式添加元素  
         aDict.update({'电话': "13456789012"})      #使用update()方法来增加元素  
         print(aDict)
```

```
{'年龄': 20, '成绩': [85, 90, 68, 72], '姓名': '王宁', 'sex': '男', '地址': '西安市长安区西京路1号', '电话': '13456789012'}
```

2. 字典的基本操作:

③ 字典元素的修改

➤ 有两种方法, 可以通过“键”的方式来赋值, 或者使用update()方法。

```
In [25]: ▶ aDict = {'年龄': 19, '成绩': [85, 90, 68, 72], '姓名': '王宁', 'sex': '男'}  
          aDict.update({"成绩": [80, 90, 68, 72]})  
          print( aDict)  
          aDict["成绩"] = [60, 70, 80, 90]  
          print( aDict)
```

```
{ '年龄': 19, '成绩': [80, 90, 68, 72], '姓名': '王宁', 'sex': '男' }  
{ '年龄': 19, '成绩': [60, 70, 80, 90], '姓名': '王宁', 'sex': '男' }
```

2. 字典的基本操作:

④ 字典元素的删除

- 删除字典元素可以使用del命令、使用字典对象的pop()和popitem()方法等方式。
- 如果要删除字典所有元素使用clear()方法。

```
In [27]: ▶ aDict = {'年龄': 19, '成绩': [85, 90, 68, 72], '姓名': '王宁', 'sex': '男'}
          del aDict["成绩"]
          print(aDict)
          aDict.pop("sex")           #删除指定键对应的元素
          print(aDict)
          aDict.popitem()           #弹出一个元素，对空字典会抛出异常
          print(aDict)
          aDict.clear()
          print(aDict)

{'年龄': 19, '姓名': '王宁', 'sex': '男'}
{'年龄': 19, '姓名': '王宁'}
{'年龄': 19}
{}
```

2. 字典的基本操作:

⑤ 通过zip()函数协助构造字典。

➤ zip()函数的作用是将可迭代对象作为参数，将对象中对应的元素打包成多个元组，然后返回由这些元组组成的对象。

```
In [27]: ▶ keys = ['ftp', 'telnet', 'www']  
values = [21, 23, 80]  
d = dict(zip(keys, values))  
print(d)  
  
{'ftp': 21, 'telnet': 23, 'www': 80}
```

3. 字典的常用方法:

方法	描述
<code>dict.get(key, default = None)</code>	返回指定键的值，若值不在字典中则返回 default
<code>dict.items()</code>	以列表返回可遍历的(键，值) 元组数组
<code>dict.keys()</code>	以列表返回一个字典所有的键
<code>dict.values()</code>	以列表返回字典中的所有值
<code>dict.has_key(key)</code>	有该键返回 TRUE ，否则 FALSE
<code>dict.update(dictict2)</code>	增加合并字典
<code>dict.popitem()</code>	得到一个 pair ，并从字典中删除它。已空则抛异常
<code>dict.clear()</code>	清空字典，同 del dict
<code>dict.copy()</code>	拷贝字典
<code>dict.cmp(dictict1,dictict2)</code>	比较字典，(优先级为元素个数、键大小、键值大小)第一个大返回 1 ，小返回 -1 ，一样返回 0

4. 对字典的转换:

➤ 可以使用函数 **tuple()** 和 **list()** 将字典转换为元组和列表，但需要注意的是，这里的转换后和之前的元素顺序是不同的，因为字典类似于散列，列表类似于链表，元组类似于列表只是元素无法改变，所以，要把散列转换为链表而顺序不变是不可行的。

4. 对字典的转换:

```
In [43]: ► fruit_dict={'apple':1, 'banana':2, 'orange':3}
dict1=tuple(fruit_dict)           # 将字典的 key 转换为元组
print(dict1)
dict2=tuple(fruit_dict.values())   # 将字典的 值 转换为元组
print(dict2)
dict3=list(fruit_dict)            # 将字典的 key 转换为列表
print(dict3)
dict4=list(fruit_dict.values())    # 将字典的 值 转换为list
print(dict4)

('apple', 'banana', 'orange')
(1, 2, 3)
['apple', 'banana', 'orange']
[1, 2, 3]
```

- 集合（set）属于Python**无序可变**序列，使用一对大括号“{}”作为定界符，元素之间使用逗号分隔，同一个集合内的每个**元素都是唯一的**，**元素之间不允许重复**。
- 集合中只能包含**数字、字符串、元组等不可变类型**（或者说可哈希）的数据，而**不能包含列表、字典、集合等可变类型的数据**。

1. 集合的创建:

- ① 直接将集合赋值给变量，即可创建一个集合对象。
- ② 也可以使用函数`set()`函数将列表、元组、字符串、`range`对象等可迭代对象转换为集合，如果原来的数据中存在重复元素，则在转换为集合的时候只保留一个；如果原序列或迭代对象中有不可哈希的值，无法转换成为集合，则抛出异常。
- ③ 创建空集合比较特殊。创建一个空集合必须用`set()`，因为`{ }`是用来创建一个空字典。

1. 集合的创建:

```
In [44]: ▶ s = {'P', 'y', 't', 'h', 'o', 'n'}  
          s1 = {1, 2, 3}                                # 整形的集合  
          s2 = {1.0, 'Python', (1, 2, 3)}              # 混合类型的集合  
          s3 = set(['P', 'y'])                          # 从列表创建  
          s4 = set()  
          print(s)  
          print(s1)  
          print(s2)  
          print(s3)  
          print(s4)  
          type(s4)  
  
          {'n', 'P', 't', 'o', 'h', 'y'}  
          {1, 2, 3}  
          {1.0, (1, 2, 3), 'Python'}  
          {'P', 'y'}  
          set()  
  
Out[44]: set
```

2. 集合的基本操作:

① 集合元素的增加:

➤使用集合对象的`add()`方法可以增加新元素，如果该元素已存在则忽略该操作，不会抛出异常；`update()`方法用于合并另外一个集合中的元素到当前集合中，并自动去除重复元素。

```
In [47]: ► s1 = {1, 2}
          s1.add(3)
          print(s1)
          s1.update([6, 4, 3])
          print(s1)
```

```
{1, 2, 3}
{1, 2, 3, 4, 6}
```

2. 集合的基本操作:

② 集合元素的删除

- `pop()` 方法用于**随机删除**并返回集合中的一个元素，如果集合为**空**则抛出**异常**。
- `remove()` 方法用于删除集合中的元素，如果指定元素**不存在**则抛出**异常**。
- `discard()` 用于从集合中删除一个特定元素，如果元素不在集合中则忽略该操作。
- `clear()` 方法清空集合删除所有元素。

2. 集合的基本操作:

② 集合元素的删除

```
In [48]: ▶ set4 = {1, 2, 3, 4}
          set4.discard(4)
          print(set4)
          set4.pop()
          print(set4)
          set4.clear()
          print(set4)
```

```
{1, 2, 3}
{2, 3}
set()
```

2. 集合的基本操作:

② 集合元素的删除

```
In [57]: ► s = {'P', 'y', 't', 'h', 'o', 'n'}
s.discard('t')          # 去掉一个存在的元素
print(s)
s.pop()                 # 随机删除一个元素
print(s)
s.discard('w')          # 去掉一个不存在的元素（正常）
print(s)
s.remove('w')           # 删除一个不存在的元素（引发错误）
print(s)
```

```
{'n', 'P', 'o', 'h', 'y'}
{'P', 'o', 'h', 'y'}
{'P', 'o', 'h', 'y'}
```

```
-----
KeyError                                Traceback (most recent call last)
<ipython-input-57-4e9043f8048c> in <module>
      6 s.discard('w')          # 去掉一个不存在的元素（正常）
      7 print(s)
----> 8 s.remove('w')          # 删除一个不存在的元素（引发错误）
      9 print(s)
```

```
KeyError: 'w'
```

2. 集合的基本操作：

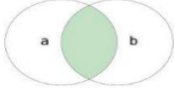
③ 集合运算

➤ Python集合支持交集、并集、差集、对称差集等运算，此外还包括集合大小的比较、子集与父集的判断等。

● 集合运算

2. 集合的基本操作：

③ 集合运算

集合运算	运算符	集合对象的方法	描述
并集	$a \mid b$	<code>a.union(b)</code>	
交集	$a \& b$	<code>a.intersection(b)</code>	
差集	$a - b$	<code>a.difference(b)</code>	
对称差集	$a \wedge b$	<code>a.symmetric_difference(b)</code>	
比较大小	$a < b, a > b, a \leq b, a \geq b$		判断集合的包含关系，返回布尔值
子集	<code>a.issubset(b)</code>		判断是否为子集合，返回布尔值
父集	<code>a.issuperset(b)</code>		判断是否为父集合，返回布尔值

2. 集合的基本操作:

③ 集合运算

➤ 使用 “&” 操作符执行交集操作，也可使用集合对象的方法intersection() 完成，。

```
In [51]: ▶ A={1, 2, 3, 4, 6, 7, 8}
          B={0, 3, 4, 5}
          print(A&B)           #用运算符求集合A和B的交集
          print(A.intersection(B)) #用函数求集合A和B的交集

          {3, 4}
          {3, 4}
```

2. 集合的基本操作:

③ 集合运算

- 使用操作符 “|” 执行并集操作，也可使用集合对象的方法 `union()` 完成。
- 使用操作符 “-” 执行差集操作，也可使用集合对象的方法 `difference()` 完成。

2. 集合的基本操作:

③ 集合运算

```
In [52]: ▶ A={1, 2, 3, 4, 6, 7, 8}
          B={0, 3, 4, 5}
          print(A|B)           #用运算符求集合A和B的并集
          print(A.union(B))    #用函数求集合A和B的并集
          print(A-B)           #用运算符求集合A和B的差集
          print(A.difference(B)) #用函数求集合A和B的差集

          {0, 1, 2, 3, 4, 5, 6, 7, 8}
          {0, 1, 2, 3, 4, 5, 6, 7, 8}
          {1, 2, 6, 7, 8}
          {1, 2, 6, 7, 8}
```

2. 集合的基本操作:

③ 集合运算

对称差: 集合A与集合B的对称差集是由只属于其中一个集合, 而不属于另一个集合的元素组成的集合, 使用“^”操作符执行对称差集操作, 也可使用集合对象的方法 `symmetric_difference()` 完成。

```
In [53]: ► A={1, 2, 3, 4, 6, 7, 8}
          B={0, 3, 4, 5}
          print(A^B)                                     #用运算符求集合A和B的对称差
          print(A.symmetric_difference(B))               #用函数求集合A和B的对称差

          {0, 1, 2, 5, 6, 7, 8}
          {0, 1, 2, 5, 6, 7, 8}
```

2. 集合的基本操作:

③ 集合运算

子集：由某个集合中一部分元素所组成的集合，使用操作符“<”判断“<”左边的集合是否是“<”右边的集合的子集，也可使用集合对象的方法issubset()完成。

```
In [55]: ▶ A={1, 2, 3, 4, 6, 7, 8}
          C={1, 3, 4}
          print(C < A)                #C集合是A集合的子集, 返回True
          print(C.issubset(A))
          print(C==A)

          True
          True
          False
```

2. 集合的基本操作:

④ 利用 `dir()` 来查看方法列表:

Python 3.7.6 Shell

File Edit Shell Debug Options Window Help

Python 3.7.6 (tags/v3.7.6:43364a7ae0, Dec 19 2019, 00:42:30) [MSC v.1916 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

```
>>> dir(set)|
```

```
['_and_', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__gt__', '__hash__', '__iand__', '__init__', '__init_subclass__', '__ior__', '__isub__', '__iter__', '__ixor__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__rand__', '__reduce__', '__reduce_ex__', '__repr__', '__ror__', '__rsub__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__xor__', 'add', 'clear', 'copy', 'difference', 'difference_update', 'discard', 'intersection', 'intersection_update', 'isdisjoint', 'issubset', 'issuperset', 'pop', 'remove', 'symmetric_difference', 'symmetric_difference_update', 'union', 'update']
```

```
>>>
```

2. 集合的基本操作:

⑤ Set各方法列表

方法	描述
<code>add()</code>	将元素添加到集合中
<code>clear()</code>	删除集合中的所有元素
<code>copy()</code>	返回集合的浅拷贝
<code>difference()</code>	将两个或多个集合的差集作为一个新集合返回
<code>difference_update()</code>	从这个集合中删除另一个集合的所有元素
<code>discard()</code>	删除集合中的一个元素（如果元素不存在，则不执行任何操作）
<code>intersection()</code>	将两个集合的交集作为一个新集合返回
<code>intersection_update()</code>	用自己 and 另一个的交集来更新这个集合
<code>isdisjoint()</code>	如果两个集合有一个空交集，返回 <code>True</code>
<code>issubset()</code>	如果另一个集合包含这个集合，返回 <code>True</code>
<code>issuperset()</code>	如果这个集合包含另一个集合，返回 <code>True</code>

2. 集合的基本操作:

⑤ set各方法列表

<code>pop()</code>	删除并返回任意的集合元素（如果集合为空，会引发 KeyError ）
<code>remove()</code>	删除集合中的一个元素（如果元素不存在，会引发 KeyError ）
<code>symmetric_difference()</code>	将两个集合的对称差作为一个新集合返回
<code>symmetric_difference_update()</code>	用自己和另一个的对称差来更新这个集合
<code>union()</code>	将集合的并集作为一个新集合返回
<code>update()</code>	用自己和另一个的并集来更新这个集合

其中一些方法在前面示例中已经被使用过了，如果有方法不会用，可利用 **help()** 函数，查看用途及详细说明。

2. 集合的基本操作：

⑥ 集合与内置函数

函数	描述
all()	如果集合中的所有元素都是 True （或者集合为空），则返回 True 。
any()	如果集合中的所有元素都是 True ，则返回 True ；如果集合为空，则返回 False 。
enumerate()	返回一个枚举对象，其中包含了集合中所有元素的索引和值（配对）。
len()	返回集合的长度（元素个数）
max()	返回集合中的最大项
min()	返回集合中的最小项
sorted()	从集合中的元素返回新的排序列表（不排序集合本身）
sum()	返回集合的所有元素之和

1. 大小比较

- 当两个列表或元组比较大小时，从第一个元素顺序开始比较，如果相等则继续。
- 返回第一个不相等元素比较的结果。
- 如果所有元素比较均相等，则长的列表大，一样长则两列表相等。

1. 大小比较

```
In [64]: ▶ a = [1, 2, 3]
          b = [1, 3, 5]
          c = [1, 2, 3, -1]
          print(a < b, a < c, b < c)
          print((1, 2, 3) == (1.0, 2.0, 3.0))
          'hello' > 'world'
```

```
True True False
True
```

```
Out[64]: False
```

2. 基本计算

- 通过内置函数`len()`、`max()`、`min()`，可以获取系列的长度、元素最大值、元素最小值。
- 内置函数`sum()`可获取列表或元组各元素之和；
- 如果有非数值元素，则导致`TypeError`；
- 对于字符串(str) 和字节数据(bytes)，也将导致`TypeError`。
- 这些基本计算函数均适用于列表、元组、字符串、集合、range对象和字典等序列结构。

● 序列的基本运算



- 通过内置函数 `all()` 和 `any()`，可以判断序列的元素是否全部和部分为True。
- `all( iterable )` 表示如果序列的所有值都为True，返回True；否则，返回False。

```
In [97]: ▶ s = 'abcdefg'
print(len(s), max(s), min(s), sep=',')
a = {1:1, 2:5, 3:8}
print(sum(a), sum(a.values()), max(a), max(a.values()), sep=',')
print(any((1, 2, 0)))
print(all([1, 2, 0]))
```

```
7, g, a
6, 14, 3, 8
True
False
```

● 总结

- 列表、字符串、元组属于有序序列，支持双向索引，支持切片操作。
- 字典和集合属于无序序列，字典可以通过“键”作为下标来访问字典中的“值”，字典不支持使用下标方式来访问。
- 列表支持在任意位置插入和删除元素，但一般建议尽量从列表的尾部进行插入和删除，这样可以获得更高的速度。
- 切片可以返回列表、元组、字符串中的部分元素，也可以对列表中的元素进行修改。

● 总结

- 列表是比数组更加方便灵活和功能强大的数据结构
- 元组属于不可变序列，一旦创建就不能修改
- 字典提供了通过键值访问数据的方式
- 集合对集合运算的支持更好